

SKILL_WEEK - 09 (SESSION NUMBER - 17)

Task 1: To implement how to list active jobs, display status information, retrieve job IDs, format output, update state information, and test job listings.

```
#include <stdio.h>
#include <string.h>

#define MAX_JOBS 5

struct Job
{
    int job_id;
    int pid;
    char status[20];
    char command[50];
};

int main()
{
    struct Job jobs[MAX_JOBS] =
    {
        {1, 1201, "RUNNING", "sleep 20"},
        {2, 1202, "STOPPED", "ping localhost"},
        {3, 1203, "RUNNING", "top"}
    };

    int count = 3;

    printf("Active Job List:\n");
    printf("-----\n");
    printf("Job ID\tPID\tStatus\t\tCommand\n");
    printf("-----\n");

    for (int i = 0; i < count; i++)
    {
        printf("%d\t%d\t%-10s\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].status,
            jobs[i].command);
    }

    strcpy(jobs[1].status, "RUNNING");

    printf("\nUpdated Job Status:\n");
    printf("Job %d status changed to %s\n",
        jobs[1].job_id,
        jobs[1].status);

    return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session17_task1.c -o bin/session17_task1
./bin/session17_task1
Active Job List:
-----
Job ID  PID    Status      Command
-----
1       1201   RUNNING     sleep 20
2       1202   STOPPED     ping localhost
3       1203   RUNNING     top

Updated Job Status:
Job 2 status changed to RUNNING
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: To design how to identify a target job, transfer terminal control, resume processes, wait for completion, update job state, and test foreground switching.

```
UW PICO 5.09

    printf("Child process completed.\n");
    exit(0);
}
else
{
    waitpid(pid, &status, WUNTRACED);

    if (WIFSTOPPED(status))
    {
        printf("Target Job PID: %d\n", pid);
        printf("Job is currently STOPPED.\n");

        printf("Transferring job to foreground...\n");

        kill(pid, SIGCONT);

        printf("SIGCONT sent to resume process.\n");

        waitpid(pid, &status, 0);

        printf("Foreground job completed.\n");
        printf("Job state updated to COMPLETED.\n");
    }
}

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % nano src/session17_task2.c
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session17_task2.c -o bin/session17_task2
[./bin/session17_task2
Child process started.
Child PID: 7932
Target Job PID: 7932
Job is currently STOPPED.
Transferring job to foreground...
SIGCONT sent to resume process.
Child process resumed in foreground.
Child process completed.
Foreground job completed.
Job state updated to COMPLETED.
amrutha@Amruthas-MacBook-Air OSSP %
```

(SESSION NUMBER - 18)

Task 1: To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    printf("\nSignal received: %d\n", sig);
}

int main()
{
    sigset_t mask;

    printf("Process ID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgid());

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    sigemptyset(&mask);
    sigaddset(&mask, SIGINT);

    sigprocmask(SIG_BLOCK, &mask, NULL);

    printf("SIGINT is temporarily blocked.\n");
    printf("Waiting for 5 seconds...\n");

    sleep(5);

    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    printf("SIGINT is now unblocked.\n");
    printf("Press Ctrl+C to test signal delivery.\n");

    while (1)
        pause();

    return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/session18_task1.c -o bin/session18_task1
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/session18_task1
Process ID: 8570
Process Group ID: 8570
SIGINT is temporarily blocked.
Waiting for 5 seconds...
SIGINT is now unblocked.
Press Ctrl+C to test signal delivery.
^C
Signal received: 2
```

Task 2: To implement how to resume stopped jobs, send continue signals, update job status, maintain the job table, verify execution, and test background recovery.

```
if (job.pid == 0)
{
    printf("Background job started.\n");
    printf("Child PID: %d\n", getpid());

    raise(SIGSTOP);

    printf("Background job resumed.\n");

    sleep(2);

    printf("Background job completed.\n");

    exit(0);
}
else
{
    strcpy(job.status, "STOPPED");

    waitpid(job.pid, &status, WUNTRACED);

    printf("\nJob Table:\n");
    printf("Job ID: %d\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("Status: %s\n", job.status);

    printf("\nSending SIGCONT...\n");

    kill(job.pid, SIGCONT);

    strcpy(job.status, "RUNNING");

    printf("Job status updated to %s\n", job.status);

    waitpid(job.pid, &status, 0);

    strcpy(job.status, "COMPLETED");

    printf("Job status updated to %s\n", job.status);
    printf("Background recovery verified successfully.\n");
}

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/session18_task2.c -o bin/session18_task2
./bin/session18_task2
Background job started.
Child PID: 9060

Job Table:
Job ID: 1
PID: 9060
Status: STOPPED

Sending SIGCONT...
Job status updated to RUNNING
Background job resumed.
Background job completed.
Job status updated to COMPLETED
Background recovery verified successfully.
```